小 型 微 型 计 算 机 系 统
Journal of Chinese Computer Systems
DOI:10.20009/j.cnki.21-1106/TP.2025-0143
2025 年 9 月 第 9 期
Vol.46 No.9 2025

非易失内存的数据库索引结构综述

蒋一赫,彭泽顺,张岩峰,于 戈

(东北大学 计算机科学与工程学院,沈阳 110169)
E-mail:zhangyf@mail.neu.edu.cn

摘 要: 非易失存储(Non-Volatile Memory,NVM)结合了动态随机访问存储器(DRAM)的高速存取特性和硬盘驱动器(HDD)或固态硬盘(SSD)的数据持久性,为传统存储和内存架构带来了革命性的变化.本文对 NVM 索引技术进行了综述,重点分析了 NVM 的关键特性和各种主流索引结构的研究进展.首先,介绍了 NVM 的基本概念、工作原理以及其在数据中心和高性能计算中的应用.接着,详细探讨了几种主要的 NVM 索引结构,包括基于哈希、B+-Tree 和 Radix-Tree 的索引结构,以及几种基于 NVM 的学习型索引和混合索引,分析了它们的设计特点、操作机制和应用场景.此外,本文还介绍了用于测试 NVM 索引性能的 benchmark 工具 PiBench,包括其设计原理、测试指标和实际应用效果.通过这些分析,本文希望为 NVM 相关领域的研究人员和技术开发者提供有价值的参考,推动 NVM 技术在实际应用中的发展和普及,最终实现更高效、更可靠的数据存储系统.

关 键 词: 非易失存储;索引结构;哈希索引;B+-Tree;Radix-Tree;学习型索引;混合索引
中图分类号: TP392 **文献标识码:** A **文 章 编 号:** 1000-1220(2025)09-2291-14

Survey on Database Index Structures on Non-volatile Memory

JIANG Yihe,PENG Zeshun,ZHANG Yanfeng,YU Ge
(School of Computer Science and Engineering,Northeastern University,Shenyang 110169,China)

Abstract: Non-Volatile Memory(NVM) combines the high-speed access characteristics of dynamic random access memory (DRAM) and the data persistence of hard disk drives (HDD) or solid-state drives (SSD),providing an alternative to traditional storage and memory architectures. It has brought about revolutionary changes. This article provides an overview of NVM index technology,focusing on analyzing the key characteristics of NVM and the research progress of various mainstream index structures. First,it introduces the basic concepts,working principles of NVM and its application in data centers. and applications in high-performance computing. Then,several main NVM index structures are discussed in detail,including index structures based on hash,B+-Tree and Radix-Tree,as well as several NVM-based learned indexes and hybrid indexes,and their design characteristics are analyzed. Operation mechanism and application scenarios. In addition,this article also introduces PiBench,a benchmark tool used to test NVM index performance,including its design principles,test indicators and practical application effects. Through these analyses,this article hopes to provide researchers and technologies in NVM-related fields with Developers provide valuable references to promote the development and popularization of NVM technology in practical applications,ultimately achieving more efficient and reliable data storage systems.

Keywords: non-volatile storage;index structure;hash index;B+tree;Radix Tree;learned index;hybrid index

0 引 言

近年来,随着数据中心和高性能计算技术的迅猛发展,存储技术和内存管理逐渐成为学术界和工业界的研究热点.非易失内存(Non-Volatile Memory,NVM)[1-3]为传统的存储和内存架构带来了革命性的变化.NVM 结合了动态随机访问存储器(Dynamic Random Access Memory,DRAM)[4]的高速存取特性和传统存储介质如硬盘驱动器(Hard Disk Drive,HDD)[5]和固态硬盘(Solid State Disk,SSD)[6]的数据持久性,在断电情况下仍能保持数据不丢失,为系统性能提升和数据持久化提供了全新的解决方案.因此,NVM 被认为是下一代存储技术的核心,具有广阔的应用前景.目前,唯一的 NVM 商用产品是 Intel 的 Optane[7]系列(傲腾持久内存 DCPMM),这款产品已在市场上得到一定应用和验证.

NVM 的引入为存储系统带来了革命性的变化,尤其是在数据持久化和访问速度方面.传统的磁盘存储和基于 DRAM 的存储体系在面对大规模数据集时,往往会受到访问延迟和能耗限制的制约.与之相比,NVM 不仅能够提供接近内存的访问速度,还能够在断电后保持数据不丢失,具有显著的优势.传统的索引设计关注的是如何快速查找数据,这一过程通常需要大量的随机读写操作,而 NVM 提供了高效的随机读写性能,理论上很适合用于索引.由于这些特性,NVM 在设计高效索引结构时具有天然的优势,尤其是在需要高并发、低延迟以及持久化的场景中.为了充分利用 NVM 的优势,研究者们探索了如何结合 NVM 的特性设计更加高效的索引结构,以应对大规模数据存储和快速查询的需求.因此,

收稿日期:2025-03-24 收修改稿日期:2025-04-29 基金项目:国家重点研发计划课题项目(2023YFB4503601)资助. 作者简介:**蒋一赫**,男,1999 年生,硕士研究生,CCF 学生会员,研究方向为基于非易失内存的数据库系统;**彭泽顺**,男,1997 年生,博士研究生,研究方向为数据库系统;**张岩峰**(通信作者),男,1982 年生,博士,教授,CCF 高级会员,研究方向为数据库、大数据处理、分布式系统;**于 戈**,男,1962 年生,博士,教授,CCF 会士,研究方向为数据库、分布式系统.

中国知网 https://www.cnki.net

索引结构在 NVM 上的应用不仅是提升数据访问性能的关键,也是实现大规模高效存储系统的必要条件. 全文共分为以下贡献:

1)NVM 的相关背景和关键特性:涵盖 NVM 的技术原理、存储介质特性、与传统存储技术的比较,以及在各类应用场景中的表现. 通过对 NVM 工作原理的详细阐述,读者可以深入理解其技术优势,如高速读写、低功耗、持久性等特性,以及这些特性如何为数据中心和高性能计算带来实际的性能提升[8].

2)介绍 NVM 索引结构的相关研究工作:涵盖多种基于 NVM 的索引方法,包括基于哈希的索引、基于 B + -Tree 的索引、基于 Radix-Tree 的索引、混合索引方法、以及基于 NVM 的学习型索引. 这部分将详细分析每种索引方法的结构特点、操作机制、适用场景以及实际应用中的性能表现.

3)介绍 NVM 索引评测软件 PiBench:评估 NVM 索引结构的性能对于深入理解和优化 NVM 索引技术至关重要. 了解 PiBench 的设计原理、测试指标和使用方法,可以更好地理解 NVM 索引结构在实际应用中的性能瓶颈和优化方向,为研究人员和工程师提供有价值的参考.

通过对 NVM 技术的深入分析和研究,本文为相关领域的研究人员和技术开发者提供参考资料,推动 NVM 技术在实际应用中的进一步发展和普及,最终实现更高效、更可靠的数据存储系统. 本文的组织结构如下:第 1 节介绍非易失内存的结构及关键特性,第 2 节介绍 NVM 索引结构,包括基于哈希索引、B + -Tree、Radix-Tree、混合索引结构的 NVM 索引,以及基于 NVM 的学习型索引,第 3 节介绍 NVM 索引结构评测工具 PiBench,第 4 节总结全文,并给出今后工作展望.

1 非易失内存

非易失内存 NVM 是一种能够在断电后仍然保留其存储数据的计算机存储技术. 与易失性内存 DRAM 不同,NVM 具有在断电后仍能保持数据不丢失的特性. NVM 主要包括相变存储器(Phase-Change Memory,PCM)[9]、忆阻器(Memristor,ReRAM)[10]、自旋扭矩转换随机存储器(Spin Transfer Torque-Magnetic Random Access Memory, STT-MRAM)[11]等. 近年来,英特尔推出了唯一的 NVM 商用产品—基于 3D Xpoint[12]技术的 Optane Memory 设备. 尽管 3D Xpoint 的具体实现细节尚未公开,但通常认为其技术基础类似于相变存储器. 以上新型非易失存储介质使用的材料各有不同,但均具备低延迟、高密度和可字节寻址的优异特性,使其在存储技术领域中占有重要地位.

NVM 结合了 DRAM 的高速存取特性与 HDD 和 SSD 的数据持久化能力,其读写速度接近 DRAM,同时能在断电后保留数据. 这使得 NVM 在高性能计算和数据密集型应用中具有巨大潜力. 如图 1 所示,包含 NVM 的计算机存储体系结构呈金字塔形,根据访问速度和容量等因素排列. 在这个层次结构中,从上到下,设备的访问时延逐渐增加,容量逐渐增大,每字节的成本则逐渐降低. 分界线以上的设备是易失性的,掉电后数据会丢失,CPU 通过 Load/Store 指令访问这些存储设备,即数据直接以缓存行(Cache Line,64 字节)的粒度在寄存器和存储设备之间进行复制. 分界线以下的设备是可持久化的存储设备,需要通过向 I/O 控制器发送命令,以块(Block)的粒度将数据缓存到 DRAM 中,然后才能被 CPU 直接访问. NVM 在存储体系层次结构中位于 DRAM 与 SSD 之间,它模糊了传统层次结构中内存与外存的边界,它既具有 DRAM 一样的数据访问方式,也具有 SSD 一样的持久化特性. 所以可以把它当成内存或者持久化的存储使用.

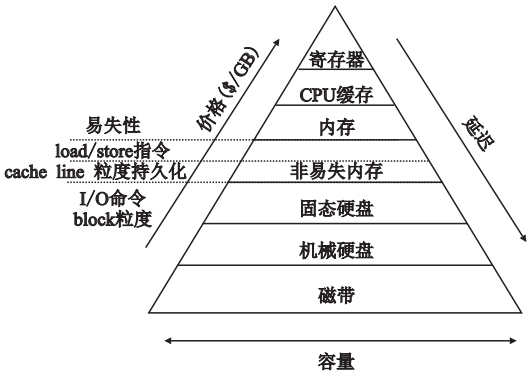

图 1 存储介质层次结构图

Fig. 1 Storage media hierarchy diagram

NVM 的结构关键特性对其相关工作如存储引擎、索引结构等的设计有重要的影响. 本节主要介绍当前唯一商用的 NVM 硬件—英特尔傲腾持久内存 DCPMM 的结构及关键特性. 在下文中,除非特别说明,否则 NVM 和 PMEM 均指代傲腾持久内存 DCPMM.

1.1 结 构

图 2 展示了 NVM 与 CPU 和 DRAM 交互的架构示意图,适用于当前和未来更新的 NVM 硬件. 两个 CPU 中,每个 CPU 都有自己的寄存器(Registers)、存储缓冲器(Store Buffer,SB)、一级缓存和二级缓存(L1,L2)和最后一级缓存(Last Level Cache,LLC). LLC 通过 Mesh Interconnect 连接,这是一种用于组件间高速数据传输的内部网络. 它支持 CPU 核心、内存控制器及其他组件之间的高效数据交换.

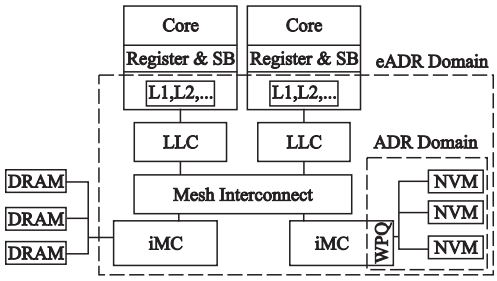

图 2 通用 NVM 系统架构图

Fig. 2 General NVM system architecture diagram

通过连接到内部互联网络上的集成内存控制器(Integrated Memory Controller, iMC),左侧为 DRAM,右侧则是 NVM. 连接 NVM 的 iMC 中包含写缓冲队列(Write Pending Queue,WPQ),其主要作用是缓冲写操作并优化内存访问,以提高系统的整体性能. 图中的 ADR 域(Asynchronous DRAM Refresh Domain)是异步 DRAM 刷新的实现,一旦数

中国知网 https://www.cnki.net

据进入 iMC 中的 WPQ,即使遭遇断电,也可持久化写入 NVM. 在第 1 代傲腾持久内存 Intel Optane PMem 100 系列中实现了 ADR 技术. 但是,ADR 架构中的 CPU 缓存并不是持久域的一部分,在遭遇断电时在 CPU 缓存中的数据可能会丢失. 增强型异步 DRAM 刷新(Enhanced Asynchronous DRAM Refresh,eADR)则扩展了 ADR 持久域,将缓存也包括在内. 值得注意的是,在出现断电故障时,寄存器和存储缓冲区中的数据仍可能丢失. Intel Optane PMem 200 系列与第 3 代 Intel 至强可扩展处理器平台提供了 eADR 的新功能,从而进一步提高了系统的可靠性和数据持久性.

图 3 中展示了傲腾持久内存的内部架构. CPU 内存控制器通过 DDR-T 协议与 DCPMM 进行通信. DDR-T 协议以高速缓存行(通常为 64B)的粒度运行,虽然其接口与 DDR4 相同,但采用了不同的通信协议,以支持异步命令和数据时序. 对 3DXPoint 介质的访问是在 256B XPLine 的较粗粒度上进行的,对于不满 256B 的写操作,这种访问模式会导致写放大效应. XPLine 还使用硬件 ECC 保护对单个 XPLine 的访问,以确保数据的完整性和可靠性. 类似 SSD,傲腾持久内存也使用逻辑寻址来实现磨损均衡,并通过地址间接表(Address Indirection Table,AIT)在内部执行地址转换,进一步优化了存储单元的寿命和性能[13].

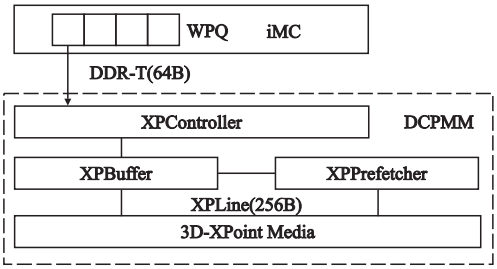

图 3　傲腾持久内存架构图

Fig. 3　Optane persistent memory architecture diagram

此外,傲腾持久内存内部还配备了带有附加预取器 XPPrefetcher 的缓存 XPBuffer,用于缓冲读取和写入操作. 这个内部缓存不仅可以组合相邻存储的写入操作,提升写入效率,而且位于 ADR 域内,这意味着任何到达 XPBuffer 的更新都是持久的,不会因断电而丢失,增强了数据写入过程中的稳定性和可靠性. 在 NVM 与虚拟存储器协同工作方面,核心在于通过地址映射、页面调度与内存映射文件等机制实现数据在 NVM 与内存之间的高效转换,虚拟存储器将进程独立地址空间与物理存储分离,通过动态页面调度实现数据载入与换出,NVM 则以非易失性和低延迟特性承担物理内存或后备存储的角色,内存映射文件使得数据可以直接访问并持久保存,操作系统与应用程序通过持久性事务和数据一致性策略保证不同介质之间的数据协调一致性.

1.2　关键特性

文献[14]和文献[15]分别对傲腾持久内存的性能进行了详细测试. 文献[14]的结果显示傲腾持久内存的性能介于 DRAM 和 SSD 之间. 其最大顺序读带宽达到了 39.4GB/s,且读性能随着线程数量的增加显著提升. 最大写带宽为13.9 GB/s,并在 4 线程时达到了峰值带宽. 同时,傲腾持久内存的随机读延迟为 305 纳秒,是 DRAM 的 3.8 倍;顺序读延迟为 169 纳秒,约为随机读延迟的一半,这表明傲腾持久内存内部的缓存机制有效优化了顺序读的效率.

文献[15]的结果显示傲腾持久内存最大顺序读带宽为 37.6GB/s,最大写带宽为 3GB/s. 随机读延迟约为 374 纳秒,大约是 DRAM 的 4 倍;写延迟约为 391 纳秒,也大概为 DRAM 的 4 倍. 尽管在延迟方面,DCPMM 的性能不及 DRAM,但其速度仍远高于毫秒级别的 SSD,使得傲腾持久内存在高性能存储解决方案中表现优异.

傲腾持久内存的主要特性包括以下几个方面:

1)字节寻址的能力:傲腾持久内存支持字节级别的 Load 和 Store 操作,而其他非易失性存储介质通常只支持速度较慢、以 Block 为单位的大数据量传输.

2)持久化特性:傲腾持久内存具有持久化特性,能够在断电后保持数据不丢失,同时提供比传统 DRAM 更高的容量和较低的延迟.

3)读写性能上的不对称性:傲腾持久内存的读带宽显著高于写带宽,且对热点数据过多的写操作可能会损坏硬件,因此在基于 NVM 的设计中需要尽量避免不必要的写操作.

4)写放大问题:傲腾持久内存中 DDR-T 的 64B 粒度和 XPLine 的 256B 粒度不一致所引起的写放大问题.

2　NVM 索引结构

在索引结构设计方面,传统的索引结构通常为 DRAM 和磁盘相结合的设计,以优化数据访问速度和存储效率. 在 DRAM 中,数据读取和写入速度极快,但断电后数据丢失,需要频繁将数据写回持久化存储介质. 磁盘虽然保证了数据的持久化,但其高延迟和块级别访问限制了数据访问速度和灵活性. 因此,传统索引结构必须在这两种存储介质之间进行平衡. 近年来,针对 NVM 设计的索引结构成为研究热点. NVM 相比 SSD 等传统存储介质具有低延迟、字节寻址和高速存取等特性,针对 NVM 设计索引存在以下挑战:

1)对 NVM 的写入会造成写放大问题.

2)NVM 支持字节级别访问,但写入延迟较高,且频繁写入 NVM 会增加存储介质磨损和能耗. 需在索引设计中进行优化,减少不必要的写操作.

3)NVM 的持久性要求索引结构能在断电或系统故障后快速恢复数据,确保数据一致性和完整性. 由于 NVM 部分写和乱序写问题,指令可能被乱序,所以不得不使用持久化指令,这将引入高的开销.

4)CPU 的原子写单元一般是 8B,如果写入超过了这个大小,就需要使用其他昂贵的技术来确保程序崩溃后数据不被破坏,例如预写日志 WAL[16],写时复制 CoW[17]等技术.

本节将介绍目前已有的基于 NVM 的索引结构,涵盖基于哈希、B+-Tree[18]和 Radix-Tree[19]的索引结构以及一些混合索引结构. 由于这些研究跨越较长时间,有些是在 DCPMM 问世之前发表的,本文将具体说明各项研究的时间背景和技术实现. 通过对这些基于 NVM 的索引结构的介绍和分析,更好地理解 NVM 在存储系统中的应用潜力和未来发展方向.

2.1　基于哈希的 NVM 索引结构

中国知网　https://www.cnki.net

哈希索引是一种基于哈希表的索引结构,通过将关键字应用哈希函数映射到哈希表的特定位置,从而实现快速的数据查找. 在哈希索引中,查找操作的时间复杂度通常为 O(1),因此在大规模数据集中具有较高的查询效率. 为了实现高速访问,NVM 经常与 CPU 的缓存一起工作. 然而,写入 NVM 的数据并不是直接存储在缓存中,而是首先进入缓存,并需要经过处理后才能写入 NVM. 在处理过程中,部分数据结果可能会暂时保存在 DRAM 或 CPU 缓存中,直到处理完成. 某些情况下,如果发生断电或系统崩溃,缓存中的数据可能尚未被持久化到 NVM,导致写丢失的问题. 此外,CPU 和内存控制器可能会对内存写入重新排序,导致数据以非预期顺序被写入 NVM,产生写乱序问题.

当把哈希索引放到 NVM 上,除将索引结构应用到 NVM 的各种挑战外,由于传统哈希索引在进行扩缩容时性能较差,需要重新分配内存并对哈希表中的所有项重新计算哈希值,导致其时间复杂度为 O(N),其中 N 为哈希表项的数量. 对哈希表进行扩容并重新计算元素的哈希值的过程称为重哈希,N 与哈希表总容量的比值则为哈希表的负载因子. 扩容过程中触发的 N 次插入操作会造成频繁的使用持久化指令,从而进一步增加系统开销. 由于 NVM 的硬件限制和数据一致性的要求,为 DRAM 设计的哈希索引在 NVM 中不再适用,因此有必要针对 NVM 特性重新设计设计哈希索引结构. 本节介绍几种有代表性的基于 NVM 的哈希索引,并针对它们的特性进行总结.

2.1.1 Level Hashing

Level Hashing[20] 针对 NVM 设计,具有写入优化、可扩展性强的特点,并提供低开销的一致性保证以及成本效益高的扩缩容机制. 其采用包含顶层和底层的双层共享结构,实现了恒定时间复杂度的操作效率. 顶层存储桶可直接寻址,而底层存储桶则用于存放从顶层逐出的冲突数据项. 此外,Level Hashing 的每个桶包含多个槽位,允许在一个桶满时最多驱逐一个数据项. 该哈希表设计使每个键有两个潜在的存储位置,通过两个散列函数计算得出(如 Cuckoo 散列[21])以提高负载系数.

在进行大小调整的过程中,Level Hashing 会创建一个容量是原表 4 倍的新哈希表,并首先将底层桶的键值对重哈希到新表中,随后清空旧底层桶的内容,新表随即成为顶层,原来的顶层转变为底层. 为了保证低开销的一致性,Level Hashing 在删除、插入及调整大小操作中采用了无日志方案,并通过每个桶头部的位图来实现. 此外,它依靠槽粒度锁进行并发控制,并在其数据结构中不维护任何指针. 虽然 Level Hashing 在多个方面优于先前的研究成果(BCH[22],PFHT[23],path hashing[24]),但它作为一种静态哈希方案,其重哈希的开销依然较大,并且未在真实 NVM 硬件上进行测试.

2.1.2 Clevel Hashing

Clevel Hashing[25] 是一种基于 Level Hashing 的无锁的高性能并发哈希表,专为崩溃一致性设计. 崩溃一致性是指在系统崩溃或断电后,确保数据的一致性和完整性. Clevel Hashing 采用多层次结构,支持并发的查询和修改操作,其调整大小的操作由后台线程异步执行,从而避免阻塞并发查询. Clevel Hashing 利用 CAS(Compare-And-Swap)原子操作实现无锁的搜索、插入、更新和删除操作,确保了操作的高效和一致性. Clevel Hashing 设计为多层结构,每层的桶数量是上一层的一半,每个桶拥有 8 个槽,每个槽 8 字节,适配缓存行大小,仅存储指向键值项的指针. 所有层通过链表形成一个层级列表,支持动态调整大小. 与 Level hashing 不同,Clevel Hashing 取消了“一步移动”机制(Level hashing 尝试通过将插入的项目复制到同一层的第 2 个候选存储桶中来对插入的项目执行一次移动,这会导致 NVM 的一次额外写入),并将槽位增加到 8 个,达到了写入优化和较高的负载因子. Clevel Hashing 通过在每次写入后立即执行缓存行刷写和内存栅栏操作确保关键数据和元数据被持久化,在重哈希过程中记录桶索引使得在故障恢复后可以从正确位置继续操作,同时采用原子分配器和无锁持久缓冲区实现内存的原子分配与回收,有效防止内存泄漏,从而保证了系统在断电或崩溃时的数据一致性.

Clevel Hashing 的动态多层设计使其可以通过添加和移除层次,在不中断查询的情况下调整大小. 搜索操作采用自下而上策略,减少了不必要的键值对比较,而插入操作在确认无重复键后,原子性地将键值对指针插入较低负载的桶中. 更新和删除操作也通过原子更新和内容感知策略来保持操作的一致性和效率.

2.1.3 CCEH

CCEH(Cacheline-Conscious Extendible Hashing)是一种 NVM 上的动态可扩展哈希[26]. CCEH 采纳了可扩展哈希的设计理念,以解决 Level Hashing 重哈希开销大的问题. 在常规的可扩展哈希方法中,目录维护存储桶地址的指针,这在存储桶数量增加时可能导致目录占用大量内存,难以完全放入 CPU 缓存中. 因此,CCEH 在目录和存储桶之间设计了“段”作为中间层,减少寻址存储桶所需的指针数量. 其将多个存储桶整合进一个段,减少寻址段所需要的位数,减少了目录的大小.

在数据插入过程中,如果目标存储桶已满并发生冲突,CCEH 不会拆分单个存储桶,而是选择拆分整个段来扩展容量. 尽管段的拆分可以降低单个存储桶的负载,但这种粗粒度的拆分可能导致空间利用率较低和增加对 NVM 的访问,因为段中的其他存储桶可能仍有空闲槽. 为了提高空间利用率,CCEH 在拆分之前采用线性探测法. 拆分段后,需要将一些项目迁移到新创建的段中. 此外,CCEH 采用了惰性删除策略,确保不需要立即清除旧段中迁移的数据,因为它们在搜索操作中会被忽略,从而避免了不必要的写入.

2.1.4 Dash

Dash[27] 在针对傲腾持久内存的特性,实现动态可扩展哈希. 具体实现为可扩展散列版本 Dash-EH 和线性散列版本 DASH-LH. 与 CCEH 相似,Dash 采用 3 层结构,包括目录和由多个桶构成的多个段. 每个目录项指向一个分段,该分段由固定数量的普通桶和用于溢出元组的存储桶组成,这些溢出记录是因为普通桶没有足够空间插入新元组.

为了解决上文中提到的 CCEH 段拆分导致空间利用率的问题,Dash 引入了平衡插入、置换和隐藏等策略. 插入过程中,首先根据键的哈希值确定段索引 s 和桶索引 b,并尝试在元数据中标记为未满的桶 b 或 b+1 中寻找空槽. 如果两个桶

中国知网 https://www.cnki.net

均已满,将触发位移操作释放空间. 成员位图在元数据中维护以加速置换过程. 若平衡插入和位移操作均无法成功执行,键将被安置到一个备用桶中. 插入成功后,会更新原始桶的溢出标记; 若失败,则进行段的拆分. 最后有两个隐藏的空间,如果这个段无法调开,那就可以利用隐藏的空间存. 这在一定程度上会减少段再扩展的频率,每个段可以相对压得更实,提高空间利用率. Dash 还使用一字节的键的简化哈希值作为指纹存储在元数据中,以提高查询的效率. Dash 采用乐观并发控制机制,并对插入操作使用桶级别的 CAS 锁确保一致性.

2.1.5　SOFT

SOFT[28] 是一个基于链式结构的无锁哈希表,其核心思想是避免在数据结构中持久化任何指针. 这个哈希表由两个核心组件组成:持久节点(PNode)和可变节点(VNode). PNode 包括一个键值对和 3 个有效状态位,用于持久化存储在 NVM 中. 与之相对的 VNode 包含一个键值对和两个指针:一个指向具有相同键值的 PNode,另一个指向下一个 VNode,所有的 VNodes 均存储在 DRAM 中.

SOFT 在进行插入操作时,系统将同时分配一个新的 PNode 和一个新的 Vnode. 新的 VNode 通过使用 CAS 操作,自动链接到已存在的 VNode 链中. 删除操作过程与插入类似,不同之处在于操作的目标是节点的移除. SOFT 的设计允许它在系统崩溃后通过扫描 NVM 中的 PNodes 并在 DRAM 中重建结构来恢复数据,这一特性提高了系统的恢复能力和数据的持久性.

2.1.6　基于哈希的 NVM 索引特性总结

数据结构:CCEH 与 Dash 采取了 3 层的目录段结构,较传统可扩展哈希更有效地节省空间. Level Hashing 则展示了双层共享结构,优化了写入效率. SOFT 基于节点链式布局,通过指针连接各节点.

并发控制:大多数哈希索引依赖于锁机制来管理并发,如 CCEH、Level Hashing. 其中 CCEH 采用多层读写锁, Level Hashing 则用槽位锁. Dash 使用了一种乐观控制策略,其写操作受桶级锁保护,读操作则无需锁,PCLHT 采用桶粒度锁定. SOFT 和 Clevel Hashing 提供了无锁操作.

架构:除了 SOFT 将易失性和持久性节点分别存于 DRAM 和 NVM 外,其他哈希表均在 NVM 中统一存储数据,这可能增加了读写操作的延迟. SOFT 的混合架构虽提供较快的访问速度,但恢复时间较长.

硬件配置:Level Hashing、CCEH 和 SOFT 的工作较早发布,由于真实的 NVM 设备尚未商用,因此并未在 DCPMM 上进行实现,而是使用 DRAM 模拟 NVM 进行测试. 而 Clevel Hashing、Dash 则是在 DCPMM 上进行了实现.

表 1　基于哈希的 NVM 索引结构

Table 1　Hash-based NVM index structure

系统	数据结构	并发控制	架构	硬件配置	动态调整策略
Level Hashing[20]	共享结构(双层)	锁(槽位粒度)	NVM	DRAM 模拟	倍增哈希表
Clevel Hashing[25]	共享结构(多层)	无锁	NVM	DCPMM	无锁动态调整
CCEH[26]	目录段结构(三层)	读写锁(多层)	NVM	DRAM 模拟	分割段
Dash[27]	目录段结构(三层)	乐观锁	NVM	DCPMM	分割段
SOFT[28]	链式结构	无锁	DRAM + NVM	DRAM 模拟	-

动态调整策略:CCEH 和 Dash 通过分割段来扩展存储空间,相比全表哈希重建减少了对 NVM 的访问. Level Hashing 通过倍增哈希表大小来调整,后者的桶共享设计允许在此过程中重用多数桶. Clevel Hashing 的动态调整则完全无锁,而 SOFT 不支持动态调整. 表 1 总结了基于哈希的 NVM 索引结构特性,图 4 给出了基于哈希的索引结构的对比图.

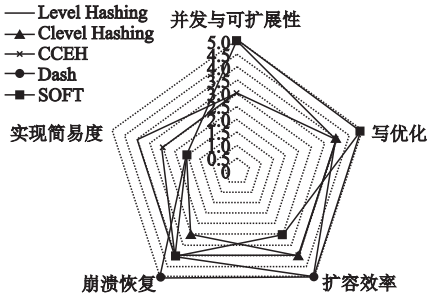

图 4　哈希索引对比图

Fig. 4　Hash index comparison chart

2.2　基于 B + -Tree 的 NVM 索引结构

B + Tree 是一种多路平衡搜索树,其核心结构特点包括以下几点. 内部节点和叶子节点分离,内部节点只存储索引键和指向下层节点的指针,而所有实际数据(或指向数据的指针)都保存在叶子节点中. 这种设计使得范围查询变得非常高效. 所有叶子节点在同一层级,保证树的高度较低,从而实现均衡查询性能. 因为所有数据都在叶子节点,所以对范围查找和顺序扫描具有较高的性能优势. 每个节点可以有多个子节点,这降低了树的高度,减少了磁盘或内存读取次数,但同时也意味着在节点更新(如插入、删除)时,可能需要对多个节点进行修改.

将 B + -Tree 索引放到 NVM 上,需要应对以下几个挑战:

1)一致性保证的高开销:由于 NVM 的部分写和乱序写问题,为确保数据一致性,必须使用持久化指令,这会引入较高的开销. B + -Tree 的复杂结构增加了保证一致性的难度.

2)写放大问题:B + -Tree 在插入和删除节点时,需要对多个节点进行更新,导致写操作放大. 由于 NVM 的写延迟高于读延迟,频繁的写操作会显著影响性能,增加系统的整体延迟.

3)节点分裂和合并的复杂性:B + -Tree 的节点分裂和合并操作需要进行大量的写操作,这在 NVM 中会导致显著的性能开销. 为了确保原子性和一致性,这些操作通常需要复杂的事务机制或日志记录,进一步增加了系统开销.

4)读写延迟不对称:真实的 NVM 设备 DCPMM 的读写延迟不对称,写延迟明显高于读延迟. 这种不对称性在 B + -Tree 的插入、删除和更新操作中表现得尤为明显,需要在设

中国知网　https://www.cnki.net

计中加以优化,以减少写操作的影响

5)持久化机制:为了在系统崩溃时保证数据的一致性,B+-Tree 需要设计复杂的持久化机制. 现有的方法包括 WAL、CoW 等,这些方法虽然可以确保数据一致性,但也带来了额外的性能开销.

本节将介绍几种有代表性的基于 B+-Tree 的 NVM 索引,并在节末对它们的特性进行总结. 我们可以看到在 NVM 上实现高效、可靠的 B+-Tree 索引结构所面临的挑战和解决方案.

2.2.1 wB^+-Trees

wB^+-Trees[29]是一种为 NVM 设计的高性能单线程 B+-Tree,其通过优化内存写入和缓存行刷新来提高效率. 传统 B+-Tree 通过节点排序以快速进行二分查找,但这种方法在插入新键时需移动数据以腾出空间,可能会在系统崩溃时导致节点状态不一致,并产生额外的内存写操作. wB^+-Trees 借鉴之前的研究,采用了无序节点策略以解决这一问题. 节点使用位图来标识每个槽的记录是否有效,新记录直接插入到空闲槽中,位图则通过一个 8 字节的写操作来原子性地更新,标记记录的有效性. 这种无序节点的使用减少了对 NVM 的写入,简化了设计,代价是查找操作由线性搜索完成,可能耗时较长.

为减少 NVM 访问,进而实现快速二分查找,wB^+-Trees 在每个节点中配置了一个间接槽数组. 该数组中每一个条目都记录了对应键的排序索引,即数组的第 n 个元素指向第 n 小的键. 此外,位图保留了一位来指示整个数组的有效性. 记录插入自由槽后,间接槽数组标记为无效并进行更新,一旦系统检测到间接槽数组无效,便在系统恢复时重建它. 位图通过原子性操作更新,确保间接槽数组和新记录的有效性. 当间接槽数组小于 8 字节,可移除位图,直接使用间接槽数组作为有效性标志,简化操作.

2.2.2 FAST&FAIR

文献[30]提出了 Failure-Atomic ShifT (FAST) 和 Failure-Atomic In-place Rebalance (FAIR),目标是解决 NVM 中 B+-Tree 的写操作的原子性问题. 在磁盘等块设备中,B+-Tree 因其高效的数据访问性能而广泛使用,但在 NVM 中,写操作的原子性粒度是 8 字节(该工作发表于 2018 年,DCPMM 尚未出现,实验环境为基于 DRAM 的 NVM 模拟器),而不是整个缓存行,这导致了粒度不匹配的问题. FAST 和 FAIR 针对 B+-Tree 的结构设计新算法,使得读操作可以容忍瞬时的不一致性,避免了使用 CoW 和日志记录,甚至无需使用读锁,从而使读事务能够非阻塞地进行.

FAST 算法的核心是确保每次 8 字节的写操作要么完成了树的一个一致状态转换,要么使树处于一个读操作可以容忍的不一致状态. 在插入和删除操作时,FAST 分别以从右向左和从左向右的顺序移动数组元素,以保持数据的一致性和正确性. 在插入操作中,算法首先在数组中找到合适的位置,然后将元素逐个向右移动,确保新元素插入后数组仍然有序. 如果系统在移动过程中崩溃,数组中可能会出现临时的重复元素,但读操作可以识别并忽略这些不一致的部分. 在删除操作中,算法从左向右移动元素,覆盖要删除的元素,同样保证数组的一致性. FAIR 算法用于解决节点分裂和合并时的原子性问题. 当节点需要分裂时,FAIR 首先创建一个兄弟节点,并将一半的条目复制到新节点中,同时更新原节点和父节点的指针,确保整个过程中树的结构保持一致. 分裂完成后,删除原节点中的冗余条目,完成分裂操作. FAIR 还支持节点合并,通过类似的方法将节点合并成一个节点,保持树的平衡.

2.2.3 NV-Tree

NV-Tree[31]是一种基于 NVM 设计的提供了一致性和缓存优化的 B+-Tree 变体,主要是为了应对传统 DRAM 在处理大内存需求时容量和性能上的限制. NV-Tree 采用了选择性一致性的策略,强化了叶节点的一致性,而对内部节点的一致性要求则较为宽松. 这种设计简化了实现流程并降低了由于频繁的缓存行刷新带来的一致性维护成本. 但在系统恢复时,内部节点需要重新构建,因为它们在 NVM 中的副本可能处于不一致状态,无法有效地指导数据查找. 由于内部节点的一致性要求较低,可以选择将它们存储在 DRAM 中,以进一步优化性能. 类似于 wB^+-Trees,NV-Tree 也引入了未排序的叶节点和追加式策略,确保了故障发生时的原子性.

内部节点的连续存储设计消除了指针的使用,提高了缓存效率. 但这种设计策略带来了在必须拆分父节点至叶节点时所需的重建过程. 为了尽量减少频繁的重建,内部节点的构建采用稀疏方式,代价是可能导致较高的内存占用. 此外,一旦内部节点构建完毕,它们就成为不可变的,除非涉及从父节点到叶节点的调整. 线程可以只在遍历树时在叶节点及其父节点级别进行锁定,从而在无锁的状态下访问这些节点. 这一设计优化了访问效率且提升了并发处理能力.

2.2.4 Bz-Tree

Bz-Tree[32]是基于 NVM 设计的无锁 B+-Tree. 它通过采用持久性多字比较并交换(Persistent Multi-word Compare-And-Swap,PMwCAS)技术优化并发处理和简化操作. PMwCAS 将 NVM 中的 8 字节定义为 word,是一种可以原子性地、无锁地更改多个 word 的高效原语,并确保操作具有崩溃一致性. 其实现包括两个阶段:第 1 阶段使用描述符收集每个目标 word 的当前值和更新值,持久化这些描述符,并为每个单词原子地关联指向描述符的指针. 如果第 1 阶段完成,则第 2 阶段对值进行更新;如遇失败,则撤销所有更改.

Bz-Tree 利用 PMwCAS 处理包括插入、删除、查询和结构调整在内的各类操作,使得多 word 的更改可以同时进行. 这种方式不仅使得 BzTree 的实现相比传统无锁设计更为直观,还因为 PMwCAS 的原子性和透明恢复特性,减少了日志记录和恢复逻辑的需求. Bz-Tree 的内部节点和叶节点都位于 NVM 中. 内部节点采用 CoW 策略,除非需要更新子节点指针,否则保持不变;而叶节点则设计为支持频繁的插入和更新操作. 新键的插入会导致父节点被新节点替换,同时必要的更新会向上至祖父节点传递,确保新的父节点被正确引用. 内部节点中的记录保持有序,以优化搜索操作;而叶节点的记录最初是无序的,通过定期排序整理,以便后续插入能继续保持记录的顺序. 这种设计理念可以优化内部节点的搜索效率和叶节点的写入性能.

2.2.5 FP-Tree

Ismail Oukid 等人提出了 FP-Tree[33],其采用了 DRAM-NVM 的混合内存架构来实现性能接近 DRAM. FP-Tree 在 DRAM 中存储内部节点,而叶节点则存放在 NVM 中. 这种配

中国知网 https://www.cnki.net

置优化了遍历性能,因为相比全 DRAM 情况,访问持久性的叶节点成本更高. 其核心理念在于即便内部节点数据丢失,也可以通过叶节点数据重建,实现了在恢复时间和运行性能之间的权衡.

为加快搜索速度,FPTree 也采用了指纹技术. 这些指纹被放置在每个叶节点的首个缓存行中. 此外,通过采用未排序的叶节点及其位图,搜索操作通过线性迭代所有有效键进行,而通过指纹扫描,叶节点中键的探查次数平均限制为一次,提升了查找效率. 在并发控制策略上,FPTree 对内部节点和叶节点采用了不同的方法,分别使用硬件事务内存(Hardware Transactional Memory,HTM)[34]和细粒度锁. 这种方法有效地解决了 HTM 与 NVM 间存在的不兼容问题,如缓存行刷新指令可能导致的 HTM 事务直接中止,提高了 FP-Tree 在处理高并发场景时的效率和稳定性.

2.2.6　LB + -Tree

LB + -Tree[35]在设计上着重优化了对 DCPMM 的利用. 考虑到其内部使用的 256 字节粒度,小于该尺寸的访问依然产生进出物理介质的 256 字节流量. 因此,精细协调 256 字节粒度的 NVM 访问并减少不必要的访问以节省带宽变得重要. LB + -Tree 吸收了 FP-Tree 中的几项关键技术,如指纹技术、DRAM + NVM 混合架构以及采用 HTM 和锁的乐观并发策略. 在此基础上,LB + -Tree 引入了新的优化措施以改善 NVM 访问效率. 它精确调整节点大小,使之与 256 字节对齐并为其倍数,这样不仅提高了 CPU 缓存效率,也优化了 NVM 的使用. 此外,为了在插入过程中最小化 NVM 写入,LB + -Tree 采取了条目移动策略,限制每次插入时的缓存行写入次数.

LB + -Tree 还采用了一种无日志的节点拆分方法来代替传统的预写日志. 通过在每个叶节点中存储两个兄弟节点指针,并在节点头部利用一个 alt 位来指示当前有效的指针,LB + -Tree 有效管理了节点的拆分. 在节点拆分过程中,首先分配并初始化一个新的叶节点,然后将部分条目移至新节点的最后两个缓存行. 最终,通过 8 字节的原子 NVM 写操作更新原节点中的 alt 位和位图,确保即使在出现故障的情况下,树的状态也能保持一致.

2.2.7　μTree

Chen 等人在 VLDB 20 上提出了 μTree[36],与大多数 NVM 索引的设计侧重点不同,μTree 主要创新点在于优化了由于 DCPMM 高延迟引起的尾延迟. 它的创新在于将 B + -Tree 的内部节点置于 DRAM 中,并对叶节点进行了重新设计,使其同时利用 DRAM 和 NVM. μTree 的叶节点结构包含两个层次:上层为一个数组,由传统的 B + -Tree 节点构成,并包含指向下层的指针; 下层则是一个位于 NVM 中的单向链表,每个链表节点存储一个键值对.

在操作上,μTree 对 DRAM 驻留的 B + -Tree 部分采用乐观锁策略,而对 NVM 驻留的链表部分实行无锁管理. 插入操作时,线程首先乐观地遍历 DRAM 中的 B + -Tree,不需要持有锁. 之后通过 CAS 操作将新的键值对节点加入到链表中,随后该线程在对应的 B + -Tree 叶节点获取锁以完成键的插入. 这一过程可能触发自底向上的树结构修改操作(Structural Modification Operations,SMOs),涉及在 DRAM 中进一步获取锁. 通过这种方式,树的 SMOs 与键值的实际插入或移除操作,即在链表层引发 NVM 刷新进行了解耦. 这允许多个线程并行地在同一叶节点的链表层中插入,提高了并发性,并有效地将 NVM 刷新从关键路径中移除,从而显著减少了尾延迟.

2.2.8　基于 B + -Tree 的索引特性总结

本节对之前讨论过的基于 B + -Tree 的各种索引进行了总结,重点关注它们在架构、节点结构、并发控制和硬件配置方面的特性,这些特性在基于 B + -Tree 的 NVM 索引设计中尤为重要.

架构:FP-Tree、NV-Tree、LB + -Tree 和 μTree 采用了将内部节点存储在 DRAM 中,叶节点存储在 NVM 中的混合架构. 这种方法在遍历到叶节点时避免了访问 NVM 的需求,实现了接近 DRAM 的性能,但在系统恢复时必须重新构建内部节点,导致恢复时间较长. 相反,wB$^+$-Trees、FAST&FAIR 和 Bz-Tree 将整个树存储在 NVM 中,可能会导致较长的查找时间.

节点结构:FAST&FAIR,μTree 的节点是有序的,文中提到的其他的树均使用未排序节点,以减少一致性成本和对 NVM 的访问,尽管这可能增加查找成本. FP-Tree 和 LB + -Tree 利用位图和指纹来优化查找性能; NV-Tree 和 Bz-Tree 则需要线性扫描未排序节点. NV-Tree 通过允许内部节点不一致来减少缓存行刷新,而 Bz-Tree 定期对叶节点进行排序合并,并通过写时复制 CoW 保持内部节点的有序性,以便使用二分查找加速遍历.

表 2　基于 B + -Tree 的 NVM 索引结构

Table 2　NVM index structure based on B + -Tree

系统	架构	节点结构	并发控制	硬件配置
wB + -Trees[29]	NVM	无序	单线程	DRAM 模拟
FAST&FAIR[30]	NVM	有序	部分无锁	DRAM 模拟
NV-tree[31]	DRAM + NVM	叶节点无序,内部节点不一致	锁	DRAM 模拟
Bz-Tree[32]	NVM	部分叶节点无序,内部节点有序	无锁(使用 PMwCAS)	DRAM 模拟
FP-Tree[33]	DRAM + NVM	叶节点无序	HTM(内部节点) + 锁(叶节点)	DRAM 模拟
LB + -Tree[35]	DRAM + NVM	叶节点无序	HTM(内部节点) + 锁(叶节点)	DCPMM
μTree[36]	DRAM + NVM	有序	乐观锁(DRAM) + 无锁(NVM)	DCPMM

并发控制:除了 wB$^+$-Trees 是单线程的,其他树采用了不同的并发控制方法. NV-Tree 使用锁,FP-Tree 和 LB + -Tree 在叶节点上使用锁,而在内部节点上使用硬件事务内存 HTM 进行选择性并发控制. μTree 在 DRAM 中的 B + -Tree 部分采用乐观锁策略,而在 NVM 中的链表部分使用无锁管理. Bz-Tree 则使用 PMwCAS 实现无锁并发控制. 需要注意的是,

中国知网　https://www.cnki.net

HTM 与 NVM 不兼容,因为缓存行刷新会直接中止事务,这一点在 FP-Tree 和 LB + -Tree 的设计中得到了体现.

硬件配置:在本节提到的所有基于 B + -Tree 的 NVM 索引中,只有 LB + -Tree 和 μTree 的实现是基于 DCPMM. 其余的工作由于发表时间较早,都是基于 DRAM 的 NVM 模拟器所实现. 表 2 总结了基于 B + -Tree 的 NVM 索引结构特性.

2.3 基于 Radix-Tree 的 NVM 索引结构

Radix-Tree 基数树是一种特殊类型的搜索树,用于存储关联数组,其中键通常是字符串或网络地址等. 与传统的二叉搜索树不同,Radix-Tree 的高度主要由存储的键的最大长度决定,而不是树中节点的数量. 这意味着即便数据量巨大,只要键的长度有限,树的高度也是有限的,这对于提高查找效率尤为重要. Radix-Tree 的一个核心特性是它无需进行树的平衡操作,因为插入的键会自然形成一定的顺序,确保任何时候插入键的顺序不会影响树的最终结构. 键在树中是按字典序排序的,从根节点到叶子节点的路径上的节点组合起来,能够重构出该叶子节点对应的键值. 相比之下,B + -Tree 等数据结构需要随着数据量的变化进行节点的分裂或合并以保持树的平衡,增加了复杂性. 图 5 展示了一棵 Radix-Tree. Radix-Tree 的一个挑战在于它对键的分布敏感,如果键的分布非常稀疏,树的内存使用效率会较低,因为许多中间节点存储的数据很少,导致空间浪费. 为优化内存使用效率,设计 Radix-Tree 时通常会采用一种策略,通过动态调整节点的结构来适应键的分布.

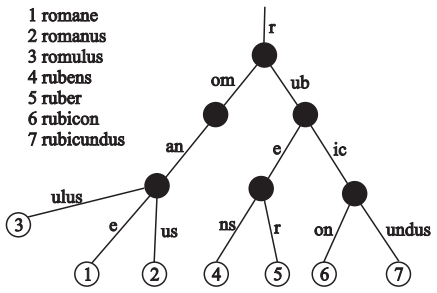

图 5 Radix-Tree

Fig. 5 Radix-Tree

ART (Adaptive Radix Tree,自适应基数树)是一种前缀树,采用高效的索引结构以适应主存数据库. 与传统的 Radix Tree 相比,ART 能够根据不同密度的键分布,自适应地选择内部节点的表示形式,从而在性能和空间利用率方面进行优化. 图 6 展示了一棵 ART. ART 的设计包括 4 种大小的内部节点,分别能够处理 4、16、48 以及 256 个键值对,以适应不同规模的数据存储需求. 内部节点在初始化时默认配置为含有 4 个数据项,这些数据项被分为键数组和子节点指针数组. 当节点中的数据量增长时,节点会根据需要进行扩展,以适应更多的数据项,从而达到下一个规定的大小. 例如,含有 48 个数据项的节点维护一个有序的键数组,子节点的指针通过 6 位索引添加到对应数组中; 而拥有 256 个数据项的节点则保持键数组的有序性,直接通过键的值快速定位到相应的子节点指针,优化了搜索效率. 这样的设计允许 ART 树以较低的内存开销高效地索引大量键值对,确保了既高效又节省空间的内存数据库索引性能. ART 树的内部节点可以采用不同的数据结构,这些结构根据子节点的数量动态变化. 例如,少量的子节点可能只会使用 Node4 或 Node16,而大量的子节点则可能会使用 Node48 或 Node256. 通过选择合适的节点类型,ART 能够在存储空间和查找性能之间取得良好的平衡. 整体而言,ART 树通过灵活的内部节点设计和高效的键值对索引,提供了优化的查找性能和优越的内存利用率,是一种适用于主存数据库的优秀索引结构.

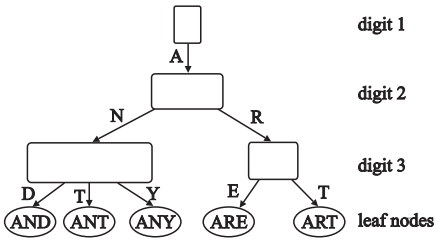

图 6 ART

Fig. 6 ART

ART 通过两种主要技术来优化性能和空间利用率. 首先是路径压缩,它可以减少表示 key 前缀的内部节点的数量,进而降低树的高度. 例如,如果一个内部节点及其所有子节点都共享相同的前缀,则这些节点可以被合并,从而减少了访问值时需要经过的节点数. 其次是懒惰扩展,这项技术可以在子节点数量较少时减少节点的创建,当新的 key 被插入并且需要新节点来区分时,再进行节点的扩展. 这些技术帮助 ART 树在处理长 key 或稀疏分布的 key 时,既减少了空间占用,又提高了搜索效率. ART 树在处理查询、插入和删除操作时都表现出了高效性,它的查找性能超过了专门为只读搜索树设计的数据结构,并且支持高效的插入和删除操作.

将 ART 设计应用于 NVM 索引时,面临诸多挑战. 首先,NVM 写操作比读取慢且具有有限的写入寿命,频繁的写操作,如 ART 的节点分裂和合并会影响 NVM 寿命和性能. 其次,ART 使用不同大小的节点动态调整节点大小,虽然有效利用内存,但查询过程中频繁的节点转换和指针追逐增加了延迟. 路径压缩虽然减少了树的高度,但处理压缩路径中的多个前缀比较操作复杂,尤其在键分布稀疏时. 由于访问多个不同大小的节点,缓存命中率降低,频繁的缓存未命中进一步增加了查询延迟,需要策略进行优化. 由于 NVM 持久性,系统崩溃后的快速恢复尤为重要,需设计机制确保系统能快速恢复到一致状态. 通过创新算法和系统优化,ART 需在这些挑战中找到平衡,以充分利用 NVM 优势,实现高性能和高可靠性的索引结构.

2.3.1 WORT

在 FAST'17 上,Lee 等人提出了 WORT (Write Optimal Radix Tree)[37],这是一种为 NVM 系统设计的高效索引结构. WORT 利用 Radix-Tree 的结构特性,避免了传统 B-Tree 变体所需的树平衡和节点粒度更新等操作,其主要贡献在于更新操作仅需一次 8 字节的原子写入即可保证结构一致性,无需复制或 CoW 来维护日志. WORT 通过失败原子的路径压缩方案优化内存使用,减少内存栅栏和缓存行刷新指令的使用,从而最小化强制故障原子性时的写入次数和指令数量. 此外,WORT 还引入了写优化自适应基数树 WOART,这是

中国知网 https://www.cnki.net

ART 的 NVM 扩展. WOART 通过为不同节点类型设计特定的一致性保证机制,实现了高效的内存使用和一致性保证,特别是对于内存中的节点类型转换和路径压缩操作,WOART 采取了故障原子性保证的修改,使其能够在 NVM 环境中高效运行.

WORT 通过在每个节点头部增添 8 字节元数据,确保发生故障时仍能保持数据一致性并优化内存空间利用效率. 元数据包含节点的深度信息、前缀长度和前缀数组. 对于前缀长度小于 6 字节的情况,前缀将直接存储于前缀数组中以便快速访问; 如果前缀长度超出存储范围,则仅在元数据中记录前缀长度,而将具体的键值比较操作延后至叶子节点执行. 此策略有效减少了非必要的内存访问,并避免了路径压缩过程中的额外复杂度.

2.3.2　ROART

Ma 等人在 FAST 21 会议上提出的 ROART (Range-query Optimized Persistent ART)[38]是一种基于 ART 的 优化范围查询的持久性索引. ROART 完全基于 NVM,选择性地保留元数据条目,并在恢复时重建不一致的条目,这类似于 NV-Tree 的选择性一致性方法.

为了优化扫描性能,ROART 将小子树(<64 个条目)压缩成存储指向记录的叶数组. 这种方法减少了扫描期间的指针追踪开销,并使索引结构更加扁平. 然而这样的设计使得节点分裂变得昂贵,因为叶数组中的所有键需要根据第一个不同的字节分成子集,然后为每个子集分配一个节点. 这过程需要更多的 NVM 写入和持久化指令,增加了 NVM 分配器的压力. 为了解决这些问题,ROART 通过放宽分裂步骤的顺序,并使用深度字段来检测和解决不一致性,从而减少了持久化指令的使用. 此外,ROART 提出了延迟检查内存管理(Delayed Check Memory Management ,DCMM) 来应对高 NVM 分配需求. DCMM 利用线程本地池进行快速分配,同时通过延迟垃圾收集来减少即时分配压力.

2.3.3　PACTree

PACTree[39]是一个仅使用 NVM 的两层持久索引结构,结合了 B+-Tree 风格的叶节点设计. PACTree 包括一个搜索层和一个数据层. 搜索层基于并发 ART,使用了读优化的写排除 (Read-Optimized Write Exclusion,ROWEX) 技术. 数据层由 B+-Tree 叶节点的双向链表组成,每个叶节点包含 64 个键值对和一个锚键,用于指示节点中最小的键. 为了优化搜索和扫描操作,PACTree 在叶节点中存储了指纹和间接数组,但这些数据不会被持久化,以减少 NVM 写入次数.

在发生节点分裂时,数据层中的目标叶节点首先被锁定,然后将日志条目写入到每个线程的 NVM 中的独立 SMO 日志中. 随后,线程分裂叶节点并提交,而不立即修改搜索层中的内部节点. 后台线程将负责完成搜索层中的剩余 SMO 操作,这使得工作线程可以在修改叶节点后尽早提交. 然而,这也会在搜索层和数据层之间产生不一致性. 因此,查询线程可能需要在到达数据层后执行搜索,利用锚键找到正确的叶节点.

2.3.4　基于 Radix-Tree 的 NVM 索引特性总结

节对之前讨论过的各种基于 Radix-Tree 的索引进行了总结,重点关注它们在架构、写入优化、查询优化、并发控制等方面的特性,这些特性在基于 Radix-Tree 的 NVM 索引设计中尤为重要.

架构:WORT 基于 Radix-Tree 设计,实现基于 NVM 模拟器; ROART 基于 ART 设计,PACTree 的设计混合了 Radix-Tree 和 B+-Tree,这两种实现都是基于真实的 NVM 硬件 DCPMM.

写入优化:WORT 通过 8 字节的原子写操作实现一致性维护,无需日志记录或 CoW,使用路径压缩减少节点数量; ROART 引入叶子节点压缩、选择性元数据持久化和最小有序分裂,减少写入次数和持久化开销; PACTree 通过路径压缩和节点合并减少写入操作和数据冗余,优化内存使用.

表 3　基于 Redix-Tree 的 NVM 索引结构

Table 3　NVM index structure based on Redix-Tree

系统	架构	并发控制	写入优化	查询优化	硬件配置
WORT[37]	NVM	无锁	8 字节原子写,路径压缩	路径压缩,自适应节点类型转换	DRAM 模拟
ROART[38]	NVM	ROWEX	叶节点压缩,选择性元数据持久化,最小有序分裂	叶节点压缩,自适应节点类型转换	DCPMM
PACTree[39]	NVM	NVM	路径压缩,节点合并	路径压缩,节点合并	DCPMM

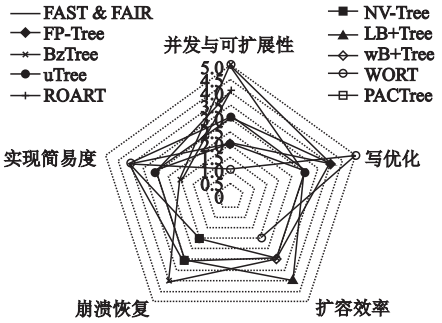

图 7　树形索引对比图

Fig. 7　Tree index comparison chart

查询优化:WORT 基于基数树结构设计,不需要键比较,前缀索引直接查找; 路径压缩和自适应节点类型转换减少树高度,优化查询路径; ROART 叶子节点压缩减少指针追逐,优化范围查询性能; 自适应节点类型转换根据查询需求调整节点大小,优化查询路径. PACTree 通过路径压缩和节点合并减少树高度和查询路径长度,结合 B+-Tree 的有序排列特性,提高查询效率. 表 3 总结了基于 Redix-Tree 的 NVM 索引结构特性. 图 7 给出了基于 B+-Tree 和 Radix-Tree 的树形索引结构对比图.

2.4　混合索引结构

基于 NVM 的独特特性,部分研究工作设计了混合索引结构. 混合索引结构通常不局限于一种存储介质,并结合了多种索引方法的优点,以充分利用 NVM 的高速存取和持久性特点,同时解决单一索引结构在性能和存储效率方面的局限. 以下介绍几种具有代表性的 NVM 混合索引结构.

2.4.1　HiKV

中国知网　https://www.cnki.net

Xia 等人在 ATC 18 中提出了 HiKV[40],这是一种新颖的混合索引键值存储方案,充分利用了 DRAM 和 NVM 的性能特征. 该方案通过构建混合索引来高效支持多种键值操作(如 Put、Get、Update、Delete 和 Scan). 具体实现上 HiKV 在 NVM 中建立哈希索引,以利用其快速索引搜索的优势,并在 DRAM 中建立 B+-Tree 索引,以支持高效的范围扫描操作,同时避免在 NVM 中进行长时间的写操作.

在 HiKV 中,哈希索引被持久化于 NVM 中,保留了其快速查找能力,并直接将索引持久化到 NVM 中,避免了从 DRAM 到 NVM 的数据复制. 另一方面,B+-Tree 索引被放置在快速但易失的 DRAM 中,主要用于加速 Scan 操作和避免由于排序和节点分裂/合并导致的 NVM 写入延迟.

为了优化索引的更新过程,HiKV 引入了异步更新策略和差异化并发控制机制. 当进行键值写操作(Put、Update、Delete)时,HiKV 首先在 NVM 中同步更新哈希索引,同时将 B+-Tree 索引的更新任务放入更新队列,由后台线程异步完成,从而隐藏了 B+-Tree 索引更新的延迟. 此外,为实现高效的并发访问,HiKV 对哈希索引进行了分区,并采用细粒度锁定来管理每个分区内的并发访问. 对于 B+-Tree 索引,HiKV 则采用了硬件事务内 HTM 进行并发控制,以确保在高并发环境下的高效操作. 在崩溃一致性方面,HiKV 仅保证哈希索引和键值项的一致性,而不保证 B+-Tree 索引的一致性,从而减少写入 NVM 的开销. 在系统恢复时,HiKV 通过扫描哈希索引来重建 B+-Tree 索引,确保数据的一致性和完整性.

2.4.2 FlatStore

Chen 等人在 ASPLOS 2020 会议上提出了 FlatStore[41],一种基于 NVM-DRAM 混合结构的 KV 存储引擎. 其使用了结合哈希表和 Masstree 索引[42]的混合索引方案. 基于 NVM 的 KV 存储面临的一个问题是,KV 存储中的小规模访问模式与 NVM 中的持久粒度不匹配,导致 NVM 带宽未得到充分利用. 为了优化写操作,FlatStore 引入了压缩日志格式,仅存储索引元数据和小型键值对,而大型键值对则由 NVM 分配器单独存储. 日志条目按追加方式存储在本地日志中,通过流水线水平批处理技术将多个小型日志条目合并为一个批次,从而减少写入操作次数,提高了 NVM 的利用效率.

FlatStore 的哈希索引 FlatStore-H 使用部署在 DRAM 中的 CCEH,目的是提供快速且高效的键值对查找. 为了支持范围查询和有序数据存储,FlatStore 引入了 Masstree 索引 FlatStore-M. 在 FlatStore-M 中,Masstree 索引用于存储键值对的元数据和指针,而实际数据存储在 NVM 中. 所有服务器核心共享一个全局的 Masstree 实例,客户端根据键的哈希值选择特定的服务器核心发送请求,减少更新 Masstree 时的可能冲突. Masstree 的多层结构允许在 DRAM 中快速进行索引,同时将实际数据存储在 NVM 中,从而在保证快速索引的同时,减少对 NVM 的写入操作.

2.4.3 DPTree

DPTree[43]是一种 DRAM-NVM 混合索引结构,结合了位于 DRAM 中的两棵 B+-Tree 作为前缓冲树和中间缓冲树,以及字典树 Trie ,其中内部节点存储在 DRAM 中,而叶节点存储在 NVM 中. 在搜索 key 时,DPTree 首先访问前缓冲树. 如果目标键未找到,则继续搜索中间缓冲树. 如果在缓冲区树中仍未找到该键,则搜索基础树. 为了减少不必要的遍历,DPTree 为每棵缓冲区树维护一个布隆过滤器. 对于范围查询,DPTree 需要搜索并合并所有树的结果.

前缓冲树和基础树之间的大小比达到预定义的阈值时,DPTree 会创建一个新的前缓冲树,并将当前的前缓冲树转变为中缓冲树. 当前缓冲树和基础树之间的大小比例达到另一个预定义阈值时,会触发树合并操作,并由后台线程执行. 通过使用版本号和额外的元数据集,DPTree 确保合并过程中并发查询不会看到未完成的更改. 合并完成后,中间缓冲树将被销毁,基础树的内部节点将被重建. 然后,全局版本比特位被改变,以反映新合并的数据结构,从而对传入请求公开更改.

2.4.4 基于 NVM 的混合索引结构总结

本节通过简述 HiKV、FlatStore 两种基于 NVM 设计的 KV 存储系统和 DPTree,介绍了各自采用的混合索引方案,展示了不同的设计和实现策略. 这些系统均采用了 DRAM 与 NVM 的混合架构,以充分利用两者的优势.

HiKV 结合了哈希表和 B+-Tree 的结构,通过分区和细粒度锁机制处理并发控制,并利用硬件事务内存 HTM 优化 B+-Tree 部分的并发性能. 此外,HiKV 在硬件配置上采用了 NVM 模拟器进行测试.

FlatStore 的混合索引结构结合了哈希表和 Masstree,通过细粒度锁机制,每个核独立锁定自己的索引实例,并结合日志结构来处理并发控制. FlatStore 在硬件配置上使用了 DCPMM.

DPTree 采用了 Trie 和 B+-Tree 的结合结构,其并发控制策略包括无锁机制(如日志记录追加)和锁机制(用于合并和垃圾回收),以实现高效并发处理. DPTree 也使用了 DCPMM. 表 4 对基于 NVM 的混合索引结构进行了总结.

2.5 基于 NVM 的学习型索引

表 4 混合索引结构

Table 4 Mixed index structure

	架构	结构	并发控制	硬件配置
HiKV[40]	DRAM+NVM	哈希+B+-Tree	分区和细粒度锁(哈希),HTM(B+-Tree)	NVM 模拟器
FlatStore[41]	DRAM+NVM	哈希+Masstree	细粒度锁(每个核锁定自己的索引实例,日志结构)	DCPMM
DPTree[43]	DRAM+NVM	Trie+B+-Tree	无锁(日志记录追加),锁(合并和垃圾回收)	DCPMM

近年来,随着人工智能的快速发展,AI 技术结合数据库系统的设计成为研究热点[44-46]. 在基于 NVM 的索引结构设计方面,探索学习型索引(Learned Index)在 NVM 环境下的表现并优化成为新的方向[46-48]. 学习型索引的核心思想是将索引结构视作一种机器学习模型,通过学习数据的分布规律,从而快速且准确地定位目标数据位置. 相较于传统的索引结构如 B+-Tree,Radix-Tree,哈希表等,学习型索引可以通过较少的内存访问来实现更高效的查询性能. 此外,由于学习型索

中国知网 https://www.cnki.net

引通常采用扁平的结构,因此可以有效降低索引层数,进一步提升数据访问速度. 然而,传统的学习型索引通常为 DRAM 环境设计,其节点规模较大、数据排序要求严格,这导致其在非易失内存 NVM 环境下存在写入性能大幅下降、写放大严重、节点结构不适合 NVM 访问模式等诸多问题. 为解决这些问题,学术界提出了一系列专门针对 NVM 设计的学习型索引结构,其中代表性的包括 APEX[49]、PLIN[50] 和 WIPE[51].

2.5.1　APEX

APEX[49] 是专为 NVM 优化的高性能学习型索引结构,基于著名的 ALEX 学习索引改进而来. APEX 着重于解决传统学习型索引在 NVM 环境下的数据移动过多和带宽瓶颈等问题. 它引入了创新的"探测与暂存(probe-and-stash)"机制,显著减少了插入操作中记录移动的次数,从而缓解了 NVM 的写入带宽瓶颈. 此外,APEX 还采用 DRAM 作为辅助结构以加快索引查询和元数据管理,并配备了轻量级并发控制机制,有效地减少了同步开销.

在具体实现方面,APEX 将数据节点视为一种特殊的哈希表,其线性模型充当了有序的哈希函数. 当模型预测相同位置出现多个键值时,会发生碰撞,传统的解决方案会导致频繁的数据移动. 为此,APEX 使用探测与暂存机制,通过在数据节点中额外设置小型暂存空间,当碰撞发生时,首先进行轻量级的探测操作寻找可用空闲位置,若探测失败,则暂存碰撞数据,从而避免了大规模的数据移动. 这种机制大幅减少了对 NVM 的不必要写入. 此外,APEX 还设计了不同大小的节点以降低结构修改操作的代价:数据节点较小以减少修改开销,内部节点较大以保持树的浅层结构,从而提升整体性能.

2.5.2　PLIN

PLIN[50] 针对纯 NVM 架构设计,致力于实现高性能查询和即时恢复的特性. PLIN 创新地提出了一种 NVM 感知的数据布局策略,通过将数据节点设计为局部无序、全局有序的结构,降低插入操作的数据移动成本. 此外,PLIN 引入了模型复制机制,将数据节点的模型参数存储在父节点中,有效避免了额外的 NVM 块访问. 同时,PLIN 还采用了一种层次化的插入策略,结合确定性插入与机会性插入,优化了节点的更新操作. 为了支持高效并发操作,PLIN 设计了细粒度锁和乐观并发控制机制.

PLIN 的层次化插入策略具体分为 3 个层级:叶节点采用确定性插入,即通过模型预测精确定位插入位置,避免搜索开销;底层的内部节点则采用机会性插入,即当有合适空间时直接插入,若空间不足则采用批量重构策略重新分配空间;而上层的内部节点不直接插入数据,而是由子节点的数据变化批量更新,以避免频繁的数据移动和重构开销. 这种策略有效降低了插入过程中的开销,提升了整体性能. 此外,PLIN 还设计了一套高效的恢复机制,以保证索引在系统崩溃后的即时可用性. 该机制通过快速重建索引的元数据结构,实现了即时恢复能力.

2.5.3　WIPE

WIPE[51] 致力于 NVM 环境下的写性能优化,提出了一种创新的 3 层结构,即根节点-索引节点-数据节点的结构. 与传统学习型索引使用大规模有序数据节点不同,WIPE 通过采用细粒度无序数据节点设计,大幅减少了插入时的数据移动操作,从而降低了写放大效应. 为保持学习型索引扁平结构的优势,WIPE 引入了一种创新的根节点构建算法,该算法通过引入中间索引节点层实现了对大量小型数据节点的快速访问. 此外,WIPE 利用 NVM 原子写特性避免了传统日志机制带来的开销,进一步优化了系统性能.

WIPE 在根节点构造过程中,通过将数据空间分割为若干小的子区间,每个子区间对应一个中间索引节点,而每个中间索引节点再分别管理若干无序数据节点. 根节点通过模型预测快速确定目标数据所在的中间索引节点,然后再由中间索引节点快速定位到具体数据节点. 这种设计巧妙地平衡了节点规模与定位效率,不牺牲读性能的前提下有效提高了写性能. 此外,WIPE 采用小型数据节点的设计,使得节点分裂操作更加轻量化,大大减少了结构修改带来的额外开销. 同时,WIPE 的原子写操作策略保证了索引数据的一致性,即使在系统故障时也能实现快速恢复.

2.5.4　基于 NVM 的学习型索引总结

本节介绍了近年来基于 NVM 的 3 种代表性学习型索引 APEX、PLIN 和 WIPE,并从数据布局、节点结构设计、并发控制策略以及即时恢复机制等多个方面进行了深入分析与对比. 在数据布局方面,APEX 采用 DRAM-NVM 混合架构,有序的节点与探测暂存机制相结合;PLIN 则采用纯 NVM 设计,局部无序全局有序的结构减少了写入时的开销;WIPE 进一步提出了完全无序的细粒度节点结构,降低了插入操作中的写放大效应.

表 5　基于 NVM 的学习型索引

Table 5　NVM-based learned index

	架构	数据节点结构	并发控制	即时恢复
APEX[49]	DRAM + NVM	有序,探测与暂存	轻量级并发控制	支持
PLIN[50]	NVM	局部无序,全局有序	乐观并发、细粒度锁	支持
WIPE[51]	NVM	无序,小型节点	原子写操作	支持

在并发控制方面,APEX 利用轻量级并发控制策略提升并发访问效率,PLIN 则通过细粒度锁和乐观并发控制策略实现高效并发操作,而 WIPE 则主要依靠原子写操作保证一致性. 此外,3 种索引都实现了即时恢复机制,保证了系统的高可靠性和快速恢复能力. 表 5 对本节介绍的基于 NVM 的学习型索引进行了总结.

3　NVM 索引结构评测工具 PiBench

评测基准工具是数据库管理系统和存储系统中不可或缺的重要组成部分,是测试与衡量系统性能的关键工具. 在本文介绍的 NVM 索引结构中,有相当一部分工作发表于 DCPMM 出现之前,基于 NVM 模拟器实现,在真实 NVM 硬件上的性能不得而知. 为了能够统一、公平地比较不同的指标,并方便以后的工作采用,需要有完备的测试工具. 截至目前,相关工作并不多,其中最具代表性的工作是 PiBench[52,53]. PiBench 为研究人员和工程师提供了一个灵活且完备的测试平台,用于全面评估和优化基于 NVM 的索引结构. 本文将从数据模型与数据生成、查询负载与执行规则、

中国知网　https://www.cnki.net

评测指标 3 个方面详细介绍 PiBench.

1)数据模型与数据生成

PiBench 允许用户自定义键和值的大小,以适应不同的应用场景,包括长键值的评估,确保基准测试结果的广泛适用性. 用户可以定义要加载和执行的记录数和操作数,确保基准测试的灵活性和可扩展性. 为了模拟不同的实际使用场景,PiBench 支持多种键分布,包括均匀分布、自相似分布和 zipfian 分布. 键值对通过随机生成,并可预先设定种子以保证测试的可重复性. 随机生成的整数经过哈希处理,避免热点键集中,确保键分布均匀.

2)查询负载与执行规则

PiBench 定义了一组通用接口,包括插入(insert)、查找(lookup)、删除(delete)、扫描(scan)和更新(update),通过共享库支持任意索引结构. 用户只需派生一个实现这些接口的类,即可将索引结构集成到 PiBench 中. 用户可以指定不同操作的比例,例如 90% 查找和 10% 更新,或者 50% 查找和 50% 插入等,以模拟不同负载下的实际使用情况. PiBench 使用多线程发出请求,评估索引结构在并发环境下的性能表现. 每个线程执行特定数量的操作,PiBench 收集所有线程的统计数据以计算总吞吐量. 测试分为加载阶段和运行阶段:在加载阶段,PiBench 会加载指定数量的记录到索引结构中,确保初始状态的一致性; 在运行阶段,执行用户定义的操作并收集和分析性能数据.

3)评测指标

PiBench 收集每个时间窗口(100 毫秒)内完成的操作数,计算平均吞吐量和标准差,作为评估索引结构性能的主要指标. 通过 Processor Counter Monitor(PCM)库和 ipmwatch,PiBench 收集硬件计数器指标,如内存访问次数、缓存未命中次数等,帮助分析硬件资源的使用情况. 专门的线程周期性地收集性能统计数据,如操作完成数,以便更好地理解性能随时间的变化. 详细的性能报告包括平均吞吐量、标准差和硬件计数器数据,使用户能够全面评估索引结构的性能. PiBench 还评估索引结构的内存消耗情况,包括 DRAM 和 NVM 的使用量,帮助用户理解索引结构的空间效率. 此外,持久分配器的性能开销是评估 NVM 索引结构的重要方面,PiBench 测试不同分配器在各种并发级别下的性能,帮助识别和优化内存分配策略.

4 总结与展望

随着 NVM 技术的快速发展,基于 NVM 的索引结构的研究成为数据存储领域的重要方向. 本文详细讨论了多种 NVM 索引结构,包括基于哈希、B+-Tree、Radix-Tree 等不同索引方法,结合机器学习的基于 NVM 的学习型索引,并分析了它们在实际应用中的性能表现和设计特点. 通过对这些研究的梳理,我们可以看到 NVM 在提高存储系统性能和可靠性方面的巨大潜力,但同时也面临着诸多挑战[54].

多样化的索引结构设计是 NVM 索引研究的核心. 本文介绍了包括基于哈希的 Level Hashing、CCEH 等,基于 B+-Tree 的 FAST&FAIR、FP-Tree 等,基于 Radix-Tree 的 WORT、PACTree 等,以及部分混合索引和基于 NVM 的学习型索引等多种索引结构. 每种方法都有其独特的设计思路和优化策略,这些结构在提高查询效率、优化写操作、保证数据一致性等方面各具特色,为 NVM 索引结构的发展提供了丰富的选择. 在设计 NVM 索引结构时,如何应对 NVM 写操作慢、写入寿命有限、断电数据恢复等问题是核心挑战[55-57]. 不同研究通过引入路径压缩、懒惰扩展、原子写操作等技术,优化了索引结构的性能和可靠性. 此外,混合架构的使用,如将内部节点存储在 DRAM 中,叶节点存储在 NVM 中,也展现了在性能和数据持久性之间的良好平衡.

本文还介绍了用于评估 NVM 索引性能的 benchmark 工具 PiBench,通过对比不同工具的设计原理和测试指标,帮助研究人员深入理解 NVM 索引结构的性能瓶颈和优化方向. 这些工具在实际应用中的表现为进一步研究和开发提供了宝贵的参考.

在最近的基于 NVM 存储引擎相关工作中,DINOMO [58]使用了基于 RECIPE[59]方法构建的哈希表作为其元数据索引结构,其关键特性在于通过所有节点共享的持久索引结构实现了高效的一致性访问. 这种索引通过在 NVM 端以异步方式合并日志条目到元数据索引中,支持无锁读取与原地更新,无需记录额外的日志,避免了节点之间的同步开销,提升了扩展性和性能?; FIR[60]采用了基于 DRAM 的 Masstree 作为索引结构以获得高吞吐性能,但通过一种基于时间戳路径标记的增量检查点机制将索引叶节点快照持久化于 NVM. 这种设计使得系统能够快速重建 DRAM 索引,从而在崩溃恢复时以较低的开销恢复索引结构,而不必牺牲运行时的性能表现?; Falcon[61]使用了 Dash 作为其索引结构之一,通过小型日志窗口机制减少 NVM 上的日志写入,并使用选择性数据刷新技术以避免缓存行粒度不匹配导致的写放大问题. 这种索引结构允许索引直接位于 NVM 上,不需频繁修改,从而实现快速的恢复,同时保持较低的 NVM 写入开销.

未来,NVM 技术及其索引结构的发展仍有许多值得探索的方向. 尽管现有研究提出了多种优化策略,但 NVM 写操作的高延迟和有限的写入寿命仍是需要重点关注的问题. 未来的研究可以继续在减少写操作、优化写路径、延长 NVM 寿命等方面深入探索. NVM 的持久性要求系统在断电或崩溃后能够快速恢复数据,确保数据一致性. 未来研究可以在设计更加高效的崩溃恢复机制、减少恢复时间、提高数据一致性等方面进行深入研究,与 AI 技术的更深层次结合也将成为未来可以探索的方向. 此外,NVM 索引结构不仅在传统的存储系统中具有应用前景,还可以扩展到更多新兴领域,如物联网、边缘计算、大数据分析等. 针对不同应用场景的需求,设计更加灵活、高效的索引结构是未来的重要研究方向. 随着硬件技术的不断进步,如高速网络、异构计算、分布式存储系统等,NVM 索引结构也需要不断演进,以充分利用新硬件的优势,进一步提升系统性能[62,63].

另一方面,傲腾持久内存的停产对 NVM 技术和相关生态系统带来了深远影响. 该产品曾为数据中心和企业级系统提供了均衡的性能和持久性支持,其退出促使业界加速向其他 NVM 技术,如 PCM、ReRAM、STT-MRAM 等的转型. 此举不仅给新型 NVM 产品的研发和商业化带来机遇,也要求开发者和研究人员重新评估和调整现有的软件工具链、仿真平

中国知网 https://www.cnki.net

台和评测标准,以适应新的硬件环境. 如何在新的平台上平衡性能、延迟和成本,并确保生态系统的平稳过渡,将成为未来亟待解决的重要课题.

未来 NVM 技术及其索引结构的研究不仅需要在写入优化、崩溃恢复和灵活索引设计等技术细节上取得突破,还必须密切关注产业动态和硬件生态的变革,以推动整个领域向更高性能、更低功耗和更长使用寿命的方向不断进步。通过持续的研究与创新,NVM 索引结构有望在更广泛的应用场景中发挥重要作用,为下一代存储技术的发展提供坚实的基础,实现更高效、更可靠的数据存储系统.

References:

[1] Han S T, Zhou Y, Roy V A L. Towards the development of flexible non-volatile memories[J]. Advanced Materials, 2013, 25(38): 5425-5449.

[2] Chen A. A review of emerging non-volatile memory (NVM) technologies and applications[J]. Solid-State Electronics, 2016, 125: 25-38, doi: 10.1016/j.sse.2016.07.006.

[3] Xue C J, Zhang Y, Chen Y, et al. Emerging non-volatile memories: opportunities and challenges[C]//Proceedings of the 7th IEEE/ACM/IFIP International Conference on Hardware/Software Codesign and System Synthesis, 2011: 325-334.

[4] Mandelman J A, Dennard R H, Bronner G B, et al. Challenges and future directions for the scaling of dynamic random-access memory (DRAM)[J]. IBM Journal of Research and Development, 2002, 46(2.3): 187-212.

[5] Yang J, Sun F B. A comprehensive review of hard-disk drive reliability[C]//Annual Reliability and Maintainability, Symposium, 1999 Proceedings(Cat. No. 99CH36283), 1999: 403-409.

[6] Caulfield A M, Mollov T I, Eisner L A, et al. Providing safe, user space access to fast, solid state disks[J]. ACM SIGPLAN Notices, 2012, 47(4): 387-400.

[7] Weiland M, Brunst H, Quintino T, et al. An early evaluation of intel's optane dc persistent memory module and its impact on high-performance scientific applications[C]//Proceedings of the International Conference for High performance Computing, Networking, Storage and Analysis, 2019: 1-19.

[8] DeBrabant J, Arulraj J, Pavlo A, et al. A prolegomenon on OLTP database systems for non-volatile memory[J]. Proceedings of the VLDB Endowment, 2014, 7(14): 57-63.

[9] Wong H S P, Raoux S, Kim S B, et al. Phase change memory[J]. Proceedings of the IEEE, 2010, 98(12): 2201-2227.

[10] Chen Y. ReRAM: history, status, and future[J]. IEEE Transactions on Electron Devices, 2020, 67(4): 1420-1433.

[11] Apalkov D, Khvalkovskiy A, Watts S, et al. Spin-transfer torque magnetic random access memory(STT-MRAM)[J]. ACM Journal on Emerging Technologies in Computing Systems, 2013, 9(2): 1-35.

[12] Hady F T, Foong A, Veal B, et al. Platform storage performance with 3D XPoint technology[J]. Proceedings of the IEEE, 2017, 105(9): 1822-1833.

[13] Gugnani S, Kashyap A, Lu X. Understanding the idiosyncrasies of real persistent memory[J]. Proceedings of the VLDB Endowment, 2020, 14(4): 626-639.

[14] Izraelevitz J, Yang J, Zhang L, et al. Basic performance measurements of the intel optane DC persistent memory module[J]. arXiv preprint arXiv: 1903.05714, 2019.

[15] Hirofuchi T, Takano R. A prompt report on the performance of intel optane dc persistent memory module[J]. IEICE Transactions on Information and Systems, 2020, 103(5): 1168-1172.

[16] Mohan C, Haderle D, Lindsay B, et al. ARIES: a transaction recovery method supporting fine-granularity locking and partial rollbacks using write-ahead logging[J]. ACM Transactions on Database Systems, 1992, 17(1): 94-162.

[17] Smith J M, Maguire Jr G Q. Effects of copy-on-write memory management on the response time of UNIX fork operations[J]. Computing Systems, 1988, 1(3): 255-278.

[18] Srinivasan V, Carey M J. Performance of B+tree concurrency control algorithms[J]. The VLDB Journal, 1993, 2(4): 361-406.

[19] Leis V, Kemper A, Neumann T. The adaptive radix tree: ARTful indexing for main-memory databases[C]//IEEE 29th International Conference on Data Engineering(ICDE), 2013: 38-49.

[20] Zuo P, Hua Y, Wu J. Write-optimized and high-performance hashing index scheme for persistent memory[C]//13th USENIX Symposium on Operating Systems Design and Implementation, 2018: 461-476.

[21] Pagh R, Rodler F F. Cuckoo hashing[J]. Journal of Algorithms, 2004, 51(2): 122-144.

[22] Fan B, Andersen D G, Kaminsky M. MemC3: compact and concurrent MemCache with dumber caching and smarter hashing[C]//10th USENIX Symposium on Networked Systems Design and Implementation, 2013: 371-384.

[23] Debnath B, Haghdoost A, Kadav A, et al. Revisiting hash table design for phase change memory[J]. ACM SIGOPS Operating Systems Review, 2016, 49(2): 18-26.

[24] Zuo P, Hua Y. A write-friendly hashing scheme for non-volatile memory systems[C]//International Conference on Massive Storage Systems and Technology, 2017: 1-10.

[25] Chen Z, Hua Y, Ding B, et al. Lock-free concurrent level hashing for persistent memory[C]//USENIX Annual Technical Conference, 2020: 799-812.

[26] Nam M, Cha H, Choi Y, et al. Write-optimized dynamic hashing for persistent memory[C]//17th USENIX Conference on File and Storage Technologies, 2019: 31-44.

[27] Lu B, Hao X, Wang T, et al. Dash: scalable hashing on persistent memory[J]. arXiv preprint arXiv: 2003.07302, 2020.

[28] Zuriel Y, Friedman M, Sheffi G, et al. Efficient lock-free durable sets[C]//Proceedings of the ACM on Programming Languages, 2019: 1-26.

[29] Chen S, Jin Q. Persistent b+-trees in non-volatile main memory[J]. Proceedings of the VLDB Endowment, 2015, 8(7): 786-797.

[30] Hwang D, Kim W H, Won Y, et al. Endurable transient inconsistency in Byte-Addressable persistent B+-Tree[C]//16th USENIX Conference on File and Storage Technologies, 2018: 187-200.

[31] Yang J, Wei Q, Chen C, et al. NV-Tree: reducing consistency cost for NVM-based single level systems[C]//13th USENIX Conference on File and Storage Technologies, 2015: 167-181.

[32] Arulraj J, Levandoski J, Minhas U F, et al. Bztree: a high-performance latch-free range index for non-volatile memory[J]. Proceedings of the VLDB Endowment, 2018, 11(5): 553-565.

[33] Oukid I, Lasperas J, Nica A, et al. FPTree: a hybrid SCM-DRAM

中国知网 https://www.cnki.net

persistent and concurrent B-tree for storage class memory[C]//Proceedings of the International Conference on Management of Data,2016:371-386.

[34] Herlihy M,Moss J E B. Transactional memory:architectural support for lock-free data structures[C]//Proceedings of the 20th Annual International Symposium on Computer Architecture,1993:289-300.

[35] Liu J,Chen S,Wang L. LB + Trees:optimizing persistent index performance on 3DXPoint memory[J]. Proceedings of the VLDB Endowment,2020,13(7):1078-1090.

[36] Chen Y,Lu Y,Fang K,et al. uTree:a persistent B + -tree with low tail latency[J]. Proceedings of the VLDB Endowment,2020,13(12):2634-2648.

[37] Lee S K,Lim K H,Song H,et al. WORT:write optimal radix tree for persistent memory storage systems[C]//15th USENIX Conference on File and Storage Technologies,2017:257-270.

[38] Ma S,Chen K,Chen S,et al. ROART:range-query optimized persistent ART[C]//19th USENIX Conference on File and Storage Technologies,2021:1-16.

[39] Kim W H,Krishnan R M,Fu X,et al. PACTree:a high performance persistent range index using PAC guidelines[C]//Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles,2021:424-439.

[40] Xia F,Jiang D,Xiong J,et al. HiKV:a hybrid index Key-Value store for DRAM-NVM memory systems[C]//USENIX Annual Technical Conference,2017:349-362.

[41] Chen Y,Lu Y,Yang F,et al. Flatstore:an efficient log-structured key-value storage engine for persistent memory[C]//Proceedings of the 25th International Conference on Architectural Support for Programming Languages and Operating Systems,2020:1077-1091.

[42] Mao Y,Kohler E,Morris R T. Cache craftiness for fast multicore key-value storage[C]//Proceedings of the 7th ACM European Conference on Computer Systems,2012:183-196.

[43] Zhou X,Shou L,Chen K,et al. DPTree:differential indexing for persistent memory[J]. Proceedings of the VLDB Endowment,2019,13(4):421-434.

[44] Chai C,Wang J,Luo Y,et al. Data management for machine learning:a survey[J]. IEEE Transactions on Knowledge and Data Engineering,2022,35(5):4646-4667.

[45] Zhou X,Sun Z,Li G. Db-gpt:large language model meets database[J]. Data Science and Engineering,2024,9(1):102-111.

[46] Zheng Y,Lin C,Lyu X,et al. Robustness of updatable learning-based index advisors against poisoning attack[J]. Proceedings of the ACM on Management of Data,2024,2(1):1-26.

[47] Li P,Hua Y,Jia J,et al. FINEdex:a fine-grained learned index scheme for scalable and concurrent memory systems[J]. Proceedings of the VLDB Endowment,2021,15(2):321-334.

[48] Wu J,Zhang Y,Chen S,et al. Updatable learned index with precise positions[J]. Proceedings of the VLDB Endowment,2021,14(8):1276-1288.

[49] Lu B,Ding J,Lo E,et al. APEX:a high-performance learned index on persistent memory[J]. Proceedings of the VLDB Endowment,2021,15(3):597-610.

[50] Zhang Z,Chu Z,Jin P,et al. PLIN:a persistent learned index for non-volatile memory with high performance and instant recovery[J]. Proceedings of the VLDB Endowment,2022,16(2):243-255.

[51] Wang Z,Ding C,Song F,et al. WIPE:a write-optimized learned index for persistent memory[J]. ACM Transactions on Architecture and Code Optimization,2024,21(2):1-25.

[52] Lersch L,Hao X,Oukid I,et al. Evaluating persistent memory range indexes[J]. Proceedings of the VLDB Endowment,2019,13(4):574-587.

[53] Hao X,Lersch L,Wang T,et al. PiBench online:interactive benchmarking of persistent memory indexes[J]. Proceedings of the VLDB Endowment,2020,13(12):2817-2820.

[54] Mishra S,Gohil B N,Ray S. A survey on persistent memory indexes:Recent advances,challenges and opportunities[J]. Journal of Systems Architecture,2024:103140,doi:10.1016/j.sysarc.2024.103140.

[55] Van Renen A,Leis V,Kemper A,et al. Managing non-volatile memory in database systems[C]//Proceedings of the International Conference on Management of Data,2018:1541-1555.

[56] Koutsoukos D,Bhartia R,Friedman M,et al. NVM:is it not very meaningful for databases?[J]. Proceedings of the VLDB Endowment,2023,16(10):2444-2457.

[57] Gao B. Emerging non-volatile memories for computation-in-memory[C]//25th Asia and South Pacific Design Automation Conference(ASP-DAC),2020:381-384.

[58] Lee S,Ponnapalli S,Singhal S,et al. Dinomo:an elastic,scalable,high-performance key-value store for disaggregated persistent memory[J]. Proceedings of the VLDB Endowment,2022,15(13):4023-4037.

[59] Lee S K,Mohan J,Kashyap S,et al. Recipe:converting concurrent dram indexes to persistent-memory indexes[C]//Proceedings of the 27th ACM Symposium on Operating Systems Principles,2019:462-477.

[60] Wei J,Zhang Q,Xiang Y,et al. FIR:achieving high throughput and fast recovery in a non-volatile memory online transaction processing engine[J]. Electronics(2079-9292),2025,14(1):39,doi:10.3390/electronics14010039.

[61] Ji Z,Chen K,Wang L,et al. Falcon:fast OLTP engine for persistent cache and non-volatile memory[C]//Proceedings of the 29th Symposium on Operating Systems Principles,2023:531-544.

[62] LIU R C,ZHANG J C,LUO Y P,et al. Heterogeneous index for non-volatile memory[J]. Journal of Software,2022,33(3):832-848.

[63] SU Q,CHEN J C,GU G S,et al. Visualization methodfor data structure evolutionin program debugging[J]. Computer Engineering,2024,50(9):197-207.

附中文参考文献:

[62] 刘睿诚,张俊晨,罗永平,等. 面向非易失内存的异构索引[J]. 软件学报,2022,33(3):832-848.

[63] 苏 庆,陈建成,顾国生,等. 程序调试中的数据结构演变可视化方法[J]. 计算机工程,2024,50(9):197-207.

中国知网 https://www.cnki.net